

T0.4 — INBOUND_WEBHOOK_SECRET obligatorio

Resumen de implementacion · Fecha: 2026-05-01 · Sin merge, sin deploy. · Rama lista para revision.

Implementado y testeado. 4 archivos modificados, 460 tests verde, branch *pr/inbound-webhook-secret-obligatorio* en GitHub. Cierra el ultimo P0 original del Plan v2.1: el endpoint `/webhook/inbound/{token}` ya no acepta tokens forjados con fallback derivado de `ADMIN_TOKEN` ni con literal publico del repo. **Phase 0 (3 P0 originales) cerrada.**

Identificadores

Item	Valor
Commit	c76219d
Mensaje	fix(seguridad): INBOUND_WEBHOOK_SECRET obligatorio en produccion
Rama local	pr/inbound-webhook-secret-obligatorio
Rama remota	origin/pr/inbound-webhook-secret-obligatorio
Crear PR en GitHub	https://github.com/celestinojbm/Dona-agent/pull/new/pr/inbound-webhook-secret-obligatorio
main local	f79b069 (post-merge T0.3; igual a origin/main, INTACTO)
origin/main	f79b069 (sin push de este PR a main)

Archivos tocados (los 4 autorizados, ninguno fuera de alcance)

Archivo	Cambio
agent/inbound_tokens.py	+ check top-level: si <code>ENVIRONMENT=production</code> y <code>INBOUND_WEBHOOK_SECRET</code> vacio (<code>post-.strip()</code>) -> <code>RuntimeError</code> aborta el deploy. Refactor de <code>_secreto()</code> : en produccion levanta <code>RuntimeError</code> si la env var desaparece despues del startup (defensa en profundidad); en dev/test mantiene los dos fallbacks (derivado de <code>ADMIN_TOKEN</code> o literal de dev) con warning. Docstring del modulo actualizado.
agent/main.py	+ import <code>agent.inbound_tokens</code> # noqa: F401 despues del import <code>agent.billing</code> ya existente. Una linea para forzar el check al startup.
.env.example	Reformulado el bloque <code>INBOUND_WEBHOOK_SECRET</code> enfatizando obligatoriedad en produccion, con explicacion del riesgo de los fallbacks. Sin valores reales.
tests/test_inbound_tokens.py	Agregada clase <code>TestSecretoProduction</code> con 5 tests: <code>RuntimeError</code> al reload sin secret, <code>RuntimeError</code> con whitespace, defensa en profundidad runtime, no levanta con secret valido, no cae a derivado de <code>ADMIN_TOKEN</code> en produccion.

Total: 4 archivos, +133 lineas, -10 lineas.

Tests corridos

Suite	Resultado
pytest tests/test_inbound_tokens.py -x -q	13 passed en 0.07 s (8 previos + 5 nuevos en TestSecretoProduction)
pytest -q (suite completa)	460 passed en 51 s, exit 0

Sin regresiones. Suite paso de 455 (post-T0.3) a 460 — +5 tests netos del PR.

Comportamiento implementado

Escenario	Antes (P0 abierto)	Ahora
ENVIRONMENT=production + INBOUND_WEBHOOK_SECRET vacio	Cae a derivado SHA256('inbound-webhook-derived' + ADMIN_TOKEN) o literal 'dona-inbound-dev-secret-do-not-use-in-prod'. Atacante forja tokens -> mensajes WhatsApp arbitrarios.	RuntimeError al import de agent.inbound_tokens -> deploy aborta antes de servir trafico.
Produccion + INBOUND_WEBHOOK_SECRET solo whitespace	Mismo fallback inseguro	RuntimeError (.strip()) lo trata como vacio).
Produccion + secret valido	Funciona correctamente	Sin cambios — _secreto() retorna los bytes correctos.
Produccion + ADMIN_TOKEN seteado, INBOUND vacio (escenario antes alto-riesgo)	Cae al derivado de ADMIN_TOKEN	RuntimeError. ADMIN_TOKEN ya NO sirve como fallback.
Dev/test sin secret	Permisivo con warning (dos fallbacks)	Permisivo con warning (mismo comportamiento, log mejorado para dejar claro 'solo dev/test').
Path runtime: secret existe al startup pero desaparece despues	No protege	Defensa en profundidad: _secreto() rechaza con RuntimeError igual.

Estado git

- **main local** en f79b069, igual a origin/main. Intacto.
- **HEAD** en pr/inbound-webhook-secret-obligatorio con c76219d.
- Sin push a main. Sin merge. Sin deploy.

Lo que NO toque (segun reglas)

- **Formato de tokens** — generar_token y verificar_token sin cambios. Solo cambia de donde sale el secret.
- **Endpoints** /admin/inbound/token y /webhook/inbound/{token} — sin cambios.
- **Rate limiting / TTL** — fuera del alcance (tareas separadas).
- Base de datos — sin cambios.
- Render / Vercel / secretos reales — sin cambios. El valor real de INBOUND_WEBHOOK_SECRET no aparece en codigo, comentarios ni tests (los tests usan dummies como 'production_secret_dummy', 'test-secret-xyz',

'admin_token_dummy').

Proximo paso recomendado (no implemento sin OK)

- **Revisar el PR en GitHub** desde el link de la portada.
- **Antes de mergear**, doble-check rapido en Render: que INBOUND_WEBHOOK_SECRET sigue presente con el valor correcto. Ya confirmaste — verificacion final.
- **Mergear** y observar Render logs en el primer deploy: si arranca normal, todo bien. Como dijiste que no hay tokens vivos conocidos, no hay riesgo de invalidacion masiva.
- Si aparece 'RuntimeError: [INBOUND] INBOUND_WEBHOOK_SECRET no configurado en produccion', revertir inmediatamente y diagnosticar la env var.
- **(Opcional, recomendado)** Probar GET /admin/inbound/token?telefono=<E.164> con tu Bearer admin -> te devuelve una URL -> hacer un POST de prueba con {'mensaje': 'test'} -> ver si llega el mensaje a WhatsApp. Si si, el flujo esta sano para futuras automatizaciones.

Cierre Phase 0 — security blockers

Con T0.4 mergeado, los **3 P0 originales** del Plan v2.1 quedan cerrados:

Tarea	Estado	Riesgo cerrado
T0.2 STRIPE_WEBHOOK_SECRET	Mergeado en main (commit 58c806b)	Acreditacion forjable de creditos
T0.3 META_APP_SECRET	Mergeado en main (commit f79b069)	Inyeccion de mensajes WhatsApp falsos via webhook Meta
T0.4 INBOUND_WEBHOOK_SECRET	Listo para merge (commit c76219d, este PR)	Mensajes WhatsApp arbitrarios via webhook inbound forjado

Pendiente nuevo (no original Phase 0)

T0.10 — whapi sin firma. Hallazgo P0 nuevo descubierto durante el diagnostico de T0.3: agent/providers/whapi.py no implementa validacion de firma de webhooks Whapi. Si WHATSAPP_PROVIDER=whapi en produccion (no es el caso hoy — vos tenes meta), un atacante que conozca la URL /webhook puede inyectar mensajes forjados. **Es tarea separada cuando lo decidas.**

Branch lista para merge. La decision queda en manos del owner.